Correlation Power Analysis in AES-128 Algorithm

6^{H} ΕΡΓΑΣΙΑ

Κωνσταντίνος Μυλωνάς | mli25036 | 03.2026

Πίνακας περιεχομένων

Εισαγωγή 3
Ζήτημα 1: Φόρτωση και Επιθεώρηση Dataset 4
Ζήτημα 2: Οπτικοποίηση των power traces 6
Ερώτηση 1: Πώς επαληθεύετε ότι τα ίχνη είναι χρονικά ευθυγραμμισμένα; 6
Ερώτηση 2: Τι παρατηρείτε όσον αφορά το θόρυβο στα πρώτα 5–10 ίχνη; 6
Ερώτηση 3: Γιατί ένα μόνο ίχνος δεν αρκεί για ανάκτηση κλειδιού; 6
Ερώτηση 4: Ποιο είναι το πλεονέκτημα της στατιστικής ανάλυσης; 6
Ζήτημα 3: Επιλογή στόχου AES για CPA επίθεση 9
Ερώτηση 1: Ποιο byte του AES επιλέγετε ως στόχο και γιατί; 9
Ερώτηση 2: Τι ρόλο παίζει το XOR μεταξύ plaintext byte και key byte; 9
Ερώτηση 3: Γιατί η έξοδος S-Box θεωρείται κλασικός στόχος; 9
Ερώτηση 4: Σε ποιον γύρο AES γίνεται η επεξεργασία; 9
Ζήτημα 4: Υλοποίηση Μοντέλου Διαρροής 11
Ερώτηση 1: Τι είναι το Hamming Weight και πώς σχετίζεται με την κατανάλωση ισχύος; 11
Ερώτηση 2: Πώς μετατρέπεται μια τιμή δεδομένων σε υποθετική ισχύ; 11
Ερώτηση 3: Πώς ελέγχετε ότι το μοντέλο HW λειτουργεί σωστά; 11
Συμπεράσματα 14

Εισαγωγή

Η κρυπτογραφία εξασφαλίζει ασφάλεια σε αλγοριθμικό επίπεδο, αλλά η φυσική υλοποίηση ενός αλγορίθμου σε πραγματικό hardware εισάγει μια διαφορετική κατηγορία απειλών. Οι επιθέσεις πλευρικού καναλιού (Side-Channel Attacks) εκμεταλλεύονται φυσικές ποσότητες που παράγει μια συσκευή κατά την εκτέλεση — όπως η κατανάλωση ισχύος, ο χρόνος εκτέλεσης και οι ηλεκτρομαγνητικές εκπομπές — για να εξάγουν μυστική πληροφορία χωρίς να επιτίθενται στον αλγόριθμο άμεσα.

Η παρούσα εργασία εστιάζει στη μέθοδο Correlation Power Analysis (CPA) εφαρμοσμένη στον αλγόριθμο AES-128, χρησιμοποιώντας προκαταγεγραμμένα ίχνη κατανάλωσης ισχύος από τον μικροελεγκτή ATmega328P. Καλύπτονται τα τέσσερα πρώτα βήματα της επίθεσης: φόρτωση και επιθεώρηση του dataset, οπτικοποίηση των ιχνών, επιλογή στόχου επίθεσης και υλοποίηση του μοντέλου διαρροής Hamming Weight.

Ζήτημα 1: Φόρτωση και Επιθεώρηση Dataset

ΕΡΩΤΗΣΗ 1: ΠΟΙΑ ΕΙΝΑΙ Η ΔΟΜΗ ΤΟΥ DATASET ATMEGA328P.HDF5;

Το αρχείο HDF5 περιέχει τρία datasets: trace με τα ίχνη κατανάλωσης ισχύος, m με τα plaintexts και c με τα ciphertexts. Κάθε γραμμή αντιστοιχεί σε μία κρυπτογράφηση.

ΕΡΩΤΗΣΗ 2: ΠΟΣΑ ΙΧΝΗ ΠΕΡΙΕΧΕΙ ΤΟ DATASET;

1.000 ίχνη (traces.shape[0] = 1000).

ΕΡΩΤΗΣΗ 3: ΠΟΣΑ ΔΕΙΓΜΑΤΑ ΕΧΕΙ ΚΑΘΕ ΙΧΝΟΣ;

132.296 δείγματα (traces.shape[1] = 132296), που καλύπτουν μία πλήρη εκτέλεση AES-128 στον ATmega328P.

ΕΡΩΤΗΣΗ 4: ΠΟΙΑ ΕΙΝΑΙ Η ΔΙΑΣΤΑΣΗ ΤΩΝ PLAINTEXTS ΚΑΙ CIPHERTEXTS;

Και τα δύο έχουν διάσταση (1000, 16) — 1000 κρυπτογραφήσεις × 16 bytes (128 bits), τύπου uint8.

ΕΡΩΤΗΣΗ 5: ΓΙΑΤΙ ΕΙΝΑΙ ΣΗΜΑΝΤΙΚΟ ΝΑ ΔΙΑΚΡΙΝΟΥΜΕ ΑΡΙΘΜΟ ΙΧΝΩΝ ΑΠΟ ΔΕΙΓΜΑΤΑ ΑΝΑ ΙΧΝΟΣ;

Τα N=1000 ίχνη είναι ο στατιστικός πληθυσμός — διαφορετικές κρυπτογραφήσεις. Τα S=132.296 δείγματα είναι η χρονική διάσταση — η εξέλιξη μιας κρυπτογράφησης στον χρόνο. Στην CPA η συσχέτιση υπολογίζεται κατά τον άξονα N για κάθε χρονικό δείγμα S χωριστά, οπότε η σύγχυση των δύο αξόνων οδηγεί σε λανθασμένα αποτελέσματα.

```python
# ─── Ζήτημα #1 – Φόρτωση Dataset ──────────────────────────
def load_dataset(filepath):
    print("\n" + "-" * 50)
    print("  ΖΗΤΗΜΑ #1 – Φόρτωση & Επιθεώρηση Dataset")
    print("-" * 50)

    with h5py.File(filepath, 'r') as f:
        print(f"Κλειδιά HDF5: {list(f.keys())}")
        traces     = np.array(f['trace'])
        plaintexts = np.array(f['m'])
        ciphers    = np.array(f['c'])

    print(f"traces     : shape={traces.shape}, dtype={traces.dtype}")
    print(f"plaintexts : shape={plaintexts.shape}, dtype={plaintexts.dtype}")
    print(f"ciphertexts: shape={ciphers.shape}, dtype={ciphers.dtype}")
    print(f"Αριθμός ιχνών  : {traces.shape[0]}")
    print(f"Δείγματα/ίχνος : {traces.shape[1]}")

    return traces, plaintexts, ciphers

```

```
--------------------------------------------------
  ΖΗΤΗΜΑ #1 – Φόρτωση & Επιθεώρηση Dataset
--------------------------------------------------
Κλειδιά HDF5: ['c', 'm', 'trace']
traces     : shape=(1000, 132296), dtype=float64
plaintexts : shape=(1000, 16), dtype=uint8
ciphertexts: shape=(1000, 16), dtype=uint8
Αριθμός ιχνών  : 1000
Δείγματα/ίχνος : 132296
```

Ζήτημα 2: Οπτικοποίηση των power traces

ΕΡΩΤΗΣΗ 1: ΠΩΣ ΕΠΑΛΗΘΕΥΕΤΕ ΟΤΙ ΤΑ ΙΧΝΗ ΕΙΝΑΙ ΧΡΟΝΙΚΑ ΕΥΘΥΓΡΑΜΜΙΣΜΕΝΑ;

Από την επικάλυψη των 10 ιχνών παρατηρούμε ότι τα χαρακτηριστικά peaks και valleys εμφανίζονται στις ίδιες χρονικές θέσεις σε κάθε ίχνος. Αυτό είναι άμεση ένδειξη χρονικής ευθυγράμμισης. Ποσοτικά, η τυπική απόκλιση ανά δείγμα (±1σ στο κάτω γράφημα) παραμένει ομοιόμορφα χαμηλή σε όλη τη χρονική έκταση, χωρίς αιφνίδιες αυξήσεις που θα υποδείκνυαν ολίσθηση. Στο dataset ATmega328P η ευθυγράμμιση εξασφαλίζεται από το hardware trigger του Picoscope 2206B που συγχρονίζει τη μέτρηση με την έναρξη κάθε κρυπτογράφησης.

ΕΡΩΤΗΣΗ 2: ΤΙ ΠΑΡΑΤΗΡΕΙΤΕ ΟΣΟΝ ΑΦΟΡΑ ΤΟ ΘΟΡΥΒΟ ΣΤΑ ΠΡΩΤΑ 5–10 ΙΧΝΗ;

Κάθε ίχνος ακολουθεί την ίδια γενική μορφή αλλά με μικρές τυχαίες αποκλίσεις από ίχνος σε ίχνος. Ο θόρυβος έχει Gaussian χαρακτήρα και είναι ανεξάρτητος μεταξύ ιχνών. Η ωφέλιμη πληροφορία (διαρροή κλειδιού) είναι παρούσα αλλά θαμμένη — δεν είναι ορατή σε μεμονωμένο ίχνος.

ΕΡΩΤΗΣΗ 3: ΓΙΑΤΙ ΕΝΑ ΜΟΝΟ ΙΧΝΟΣ ΔΕΝ ΑΡΚΕΙ ΓΙΑ ΑΝΑΚΤΗΣΗ ΚΛΕΙΔΙΟΥ;

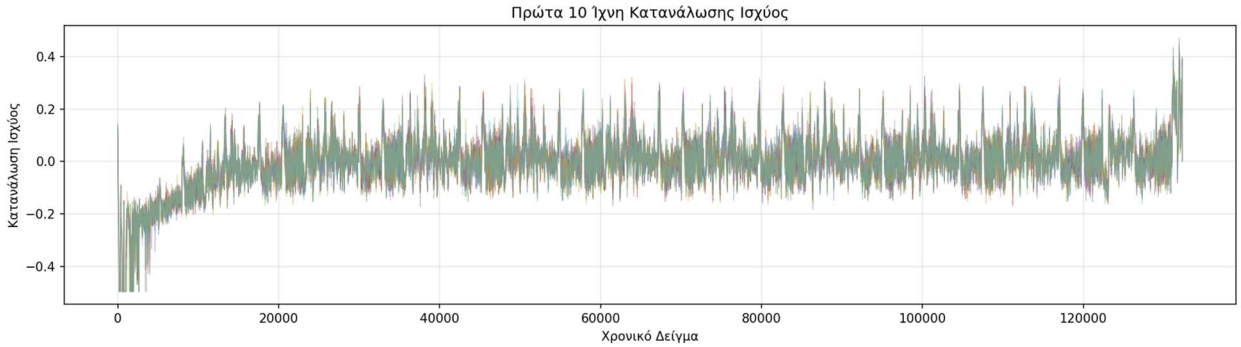
Η ενεργειακή διαρροή που εξαρτάται από το κλειδί είναι πολύ μικρή σε σχέση με τον θόρυβο μέτρησης. Με ένα ίχνος δεν μπορούμε να διακρίνουμε αν μια αιχμή οφείλεται στο κλειδί ή στον θόρυβο. Το SNR βελτιώνεται ως $O(\sqrt{N})$, οπότε με $N=1000$ ίχνη η βελτίωση είναι ~31× σε σχέση με ένα μεμονωμένο ίχνος.

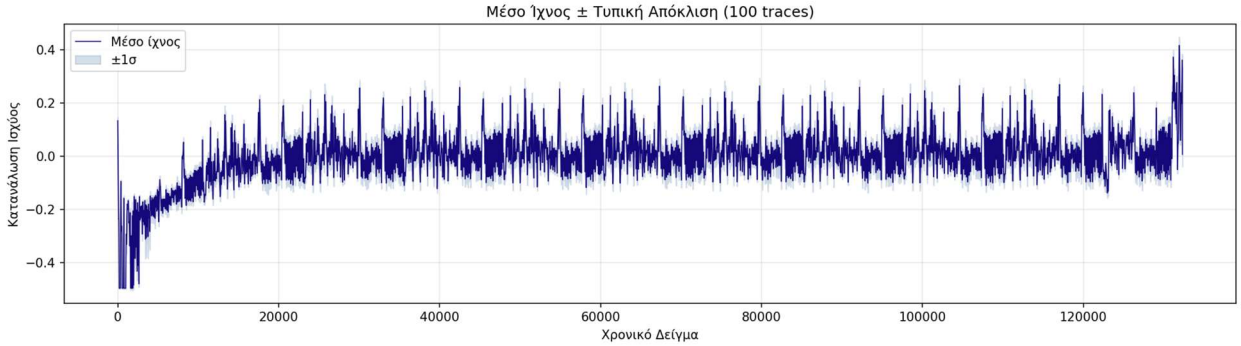
ΕΡΩΤΗΣΗ 4: ΠΟΙΟ ΕΙΝΑΙ ΤΟ ΠΛΕΟΝΕΚΤΗΜΑ ΤΗΣ ΣΤΑΤΙΣΤΙΚΗΣ ΑΝΑΛΥΣΗΣ;

Ο συντελεστής συσχέτισης Pearson ενισχύει το συστηματικό σήμα διαρροής ενώ ο τυχαίος θόρυβος ακυρώνεται με τη συσσώρευση ιχνών. Για τη σωστή υπόθεση κλειδιού ο συντελεστής αποκλίνει σημαντικά από το μηδέν· για λανθασμένες υποθέσεις παραμένει κοντά στο μηδέν. Αυτό επιτρέπει αξιόπιστη αναγνώριση του σωστού κλειδιού μέσα από 256 ανταγωνιστικές υποθέσεις.

```python
# ─── Ζήτημα #2 — Οπτικοποίηση Ιχνών ──────────────────────────
def visualize_traces(traces):
    print("\n" + "─" * 50)
    print("  ΖΗΤΗΜΑ #2 — Οπτικοποίηση Ιχνών")
    print("─" * 50)

    fig, axes = plt.subplots(2, 1, figsize=(14, 8))

    for i in range(10):
        axes[0].plot(traces[i], alpha=0.4, linewidth=0.6)
    axes[0].set_title('Πρώτα 10 Ίχνη Κατανάλωσης Ισχύος')
    axes[0].set_xlabel('Χρονικό Δείγμα')
    axes[0].set_ylabel('Κατανάλωση Ισχύος')
    axes[0].grid(True, alpha=0.3)

    mean_trace = np.mean(traces[:100], axis=0)
    std_trace  = np.std(traces[:100],  axis=0)
    x = np.arange(len(mean_trace))
    axes[1].plot(mean_trace, color='navy', linewidth=0.8, label='Μέσο ίχνος')
    axes[1].fill_between(x,
                         mean_trace - std_trace,
                         mean_trace + std_trace,
                         alpha=0.25, color='steelblue', label='±1σ')
    axes[1].set_title('Μέσο Ίχνος ± Τυπική Απόκλιση (100 traces)')
    axes[1].set_xlabel('Χρονικό Δείγμα')
    axes[1].set_ylabel('Κατανάλωση Ισχύος')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
    path = os.path.join(OUTPUT_DIR, "fig1_power_traces.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"Αποθηκεύτηκε: {path}")

```

```
──────────────────────────────────────────────────
  ΖΗΤΗΜΑ #2 — Οπτικοποίηση Ιχνών
──────────────────────────────────────────────────
Αποθηκεύτηκε: figures_project6\fig1_power_traces.png
```


Πρώτα 10 Ίχνη Κατανάλωσης Ισχύος
Κατανάλωση Ισχύος
Χρονικό Δείγμα


Μέσο Ίχνος ± Τυπική Απόκλιση (100 traces)
Μέσο ίχνος
±1σ
Κατανάλωση Ισχύος
Χρονικό Δείγμα

Ζήτημα 3: Επιλογή στόχου AES για CPA επίθεση

ΕΡΩΤΗΣΗ 1: ΠΟΙΟ BYTE ΤΟΥ AES ΕΠΙΛΕΓΕΤΕ ΩΣ ΣΤΟΧΟ ΚΑΙ ΓΙΑΤΙ;

Επιλέγεται το πρώτο byte (byte_idx = 0) της εξόδου του S-Box στον 10 γύρο. Σε πλήρη CPA επίθεση η διαδικασία επαναλαμβάνεται 16 φορές — μία για κάθε byte κλειδιού — για να ανακτηθεί ολόκληρο το 128-bit κλειδί.

ΕΡΩΤΗΣΗ 2: ΤΙ ΡΟΛΟ ΠΑΙΖΕΙ ΤΟ XOR ΜΕΤΑΞΥ PLAINTEXT BYTE ΚΑΙ KEY BYTE;

Το XOR αντιστοιχεί στη λειτουργία AddRoundKey — την πρώτη πράξη του AES. Είναι η μοναδική σύνδεση μεταξύ του γνωστού plaintext και του άγνωστου κλειδιού, και αυτό επιτρέπει τη διατύπωση υποθέσεων: για κάθε πιθανό key_byte (0–255) υπολογίζουμε την ενδιάμεση τιμή και προβλέπουμε την αντίστοιχη κατανάλωση ισχύος.

ΕΡΩΤΗΣΗ 3: ΓΙΑΤΙ Η ΕΞΟΔΟΣ S-BOX ΘΕΩΡΕΙΤΑΙ ΚΛΑΣΙΚΟΣ ΣΤΟΧΟΣ;

Διότι βρίσκεται αμέσως μετά το AddRoundKey, εξαρτάται άμεσα από XOR(plaintext, key), και παράγεται νωρίς στην εκτέλεση πριν η πληροφορία διαχυθεί μέσω ShiftRows και MixColumns. Επιπλέον, η υλοποίηση του S-Box ως lookup table σε 8-bit μικροελεγκτή προκαλεί ισχυρό και αναγνωρίσιμο ενεργειακό αποτύπωμα.

ΕΡΩΤΗΣΗ 4: ΣΕ ΠΟΙΟΝ ΓΥΡΟ AES ΓΙΝΕΤΑΙ Η ΕΠΕΞΕΡΓΑΣΙΑ;

Στον 10 γύρο, στη λειτουργία SubBytes, αμέσως μετά το αρχικό AddRoundKey. Σε αυτό το σημείο η σχέση plaintext–key είναι άμεση και δεν έχει ακόμα διαχυθεί από τις επόμενες λειτουργίες.

```python
# ── Ζήτημα #3 — Επιλογή Στόχου AES ──────────────────────────
def target_demo(plaintexts):
    print("\n" + "—" * 50)
    print("  ΖΗΤΗΜΑ #3 — Επιλογή Στόχου AES")
    print("—" * 50)

    byte_idx   = 0
    example_pt = plaintexts[0, byte_idx]
    key_byte   = 0x2B
    xor_val    = int(example_pt) ^ key_byte
    sbox_out   = SBOX[xor_val]
    hw         = bin(sbox_out).count('1')

    print(f"Στόχος : v = SBox( plaintext[{byte_idx}] XOR key[{byte_idx}] )")
    print(f"Γύρος  : 1ος — SubBytes αμέσως μετά AddRoundKey")
    print(f"plaintext[0][0] = 0x{example_pt:02X}")
    print(f"key[0]          = 0x{key_byte:02X}")
    print(f"XOR             = 0x{xor_val:02X}")
    print(f"S-Box output    = 0x{sbox_out:02X}")
    print(f"Hamming Weight  = {hw}")

```

```
────────────────────────────────────────────────────────────
  ΖΗΤΗΜΑ #3 — Επιλογή Στόχου AES
────────────────────────────────────────────────────────────

Στόχος : v = SBox( plaintext[0] XOR key[0] )
Γύρος  : 1ος — SubBytes αμέσως μετά AddRoundKey
plaintext[0][0] = 0x29
key[0]          = 0x2B
XOR             = 0x02
S-Box output    = 0x77
Hamming Weight  = 6
```

Ζήτημα 4: Υλοποίηση Μοντέλου Διαρροής

ΕΡΩΤΗΣΗ 1: ΤΙ ΕΙΝΑΙ ΤΟ HAMMING WEIGHT ΚΑΙ ΠΩΣ ΣΧΕΤΙΖΕΤΑΙ ΜΕ ΤΗΝ ΚΑΤΑΝΑΛΩΣΗ ΙΣΧΥΟΣ;

Το Hamming Weight μιας τιμής x είναι ο αριθμός των bits με τιμή 1. Σε κυκλώματα CMOS κάθε bit '1' αντιστοιχεί σε φορτισμένο transistor που καταναλώνει ενέργεια, οπότε τιμές με υψηλό HW αναμένεται να παράγουν υψηλότερη κατανάλωση ισχύος κατά τη φόρτιση του data bus.

ΕΡΩΤΗΣΗ 2: ΠΩΣ ΜΕΤΑΤΡΕΠΕΤΑΙ ΜΙΑ ΤΙΜΗ ΔΕΔΟΜΕΝΩΝ ΣΕ ΥΠΟΘΕΤΙΚΗ ΙΣΧΥ;

Για κάθε υπόθεση κλειδιού k ∈ {0,…,255} και για κάθε ίχνος i υπολογίζουμε:

- intermediate = SBOX[plaintext[i][0] XOR k]
- hypothetical_power = HW(intermediate)

Το αποτέλεσμα αποθηκεύεται σε πίνακα σχήματος (256, N), όπου κάθε γραμμή αντιστοιχεί σε μία υπόθεση κλειδιού.

ΕΡΩΤΗΣΗ 3: ΠΩΣ ΕΛΕΓΧΕΤΕ ΟΤΙ ΤΟ ΜΟΝΤΕΛΟ HW ΛΕΙΤΟΥΡΓΕΙ ΣΩΣΤΑ;

Με unit testing σε τιμές γνωστού αποτελέσματος, όπως φαίνεται στον πίνακα επαλήθευσης. Επιπλέον, ο μέσος HW για όλες τις εξόδους S-Box είναι 4.000 — ακριβώς το αναμενόμενο μέσο μιας binomial(8, 0.5) — και η κατανομή στο γράφημα έχει κανονική μορφή με κορυφή στο 4, επιβεβαιώνοντας τη σωστή λειτουργία.

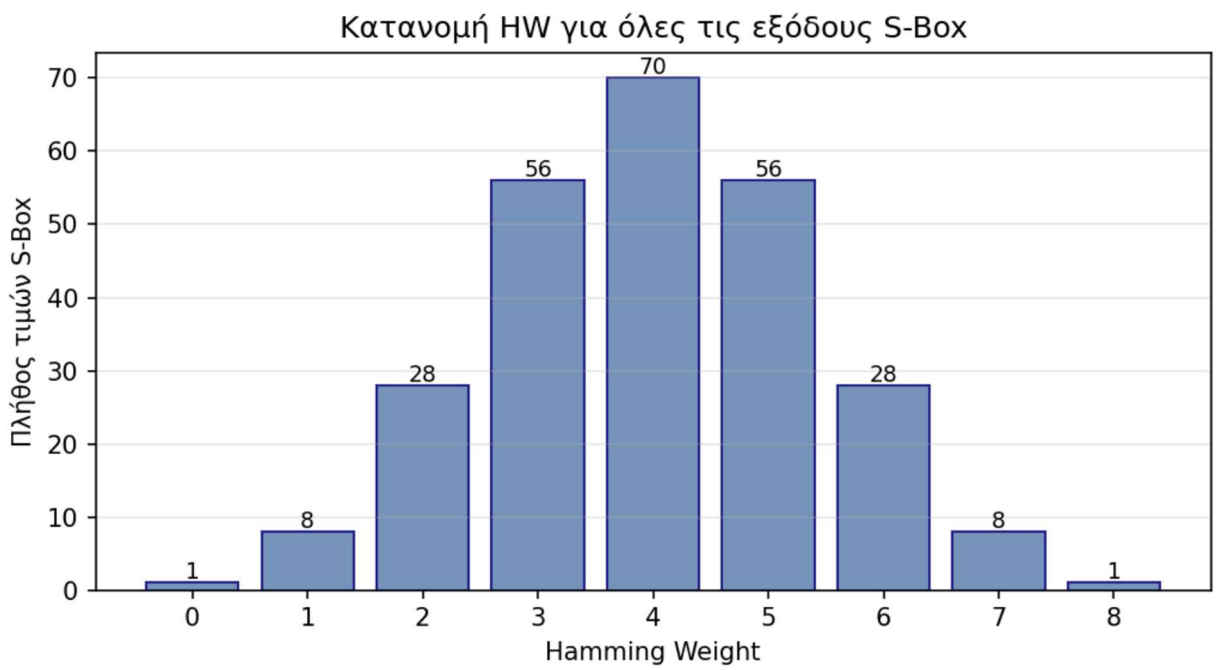
```
# ── Ζήτημα #4 – Μοντέλο Διαρροής (Hamming Weight) ──────────
def hamming_weight(x):
    if isinstance(x, np.ndarray):
        result = np.zeros(x.shape, dtype=np.uint8)
        tmp = x.astype(np.uint32)
        while tmp.any():
            result += (tmp & 1).astype(np.uint8)
            tmp >>= 1
        return result
    else:
        count = 0
        while x:
            count += x & 1
            x >>= 1
        return count
```

```python
def verify_and_plot_hw():
    print("\n" + "—" * 50)
    print("  ΖΗΤΗΜΑ #4 — Μοντέλο Διαρροής (Hamming Weight)")
    print("—" * 50)

    # Επαλήθευση
    known = {0x00: 0, 0x01: 1, 0x0F: 4, 0xFF: 8, 0x55: 4, 0xAA: 4}
    print(f"\n{'Τιμή':>6}  {'Αναμ.':>6}  {'Υπολ.':>6}  {'OK':>4}")
    for val, expected in known.items():
        computed = hamming_weight(val)
        ok = "✓" if computed == expected else "✗"
        print(f"0x{val:02X}    {expected:>6}  {computed:>6}  {ok}")

    # Στατιστικά
    hw_vals = np.array([hamming_weight(int(SBOX[i])) for i in range(256)])
    print(f"\nΜέσος HW   : {hw_vals.mean():.3f}")
    print(f"Τυπ. απόκλ.: {hw_vals.std():.3f}")

    # Γράφημα
    counts = np.bincount(hw_vals, minlength=9)
    plt.figure(figsize=(7, 4))
    plt.bar(range(9), counts, color='steelblue', edgecolor='navy', alpha=0.85)
    plt.xlabel('Hamming Weight')
    plt.ylabel('Πλήθος τιμών S-Box')
    plt.title('Κατανομή HW για όλες τις εξόδους S-Box')
    plt.xticks(range(9))
    plt.grid(axis='y', alpha=0.3)
    for i, c in enumerate(counts):
        if c > 0:
            plt.text(i, c + 0.5, str(c), ha='center', fontsize=9)
    plt.tight_layout()
    path = os.path.join(OUTPUT_DIR, "fig2_hw_distribution.png")
    plt.savefig(path, dpi=150)
    plt.close()
    print(f"Αποθηκεύτηκε: {path}")
```

(function) edgecolor: Literal['navy']

```python
# ── Εκτέλεση ──────────────────────────────────────────────
if __name__ == "__main__":
    traces, plaintexts, ciphers = load_dataset('atmega328p.hdf5')
    visualize_traces(traces)
    target_demo(plaintexts)
    verify_and_plot_hw()

    print("\n" + "—" * 50)
    print(f"  Ολοκλήρωση — γραφήματα στον φάκελο: {OUTPUT_DIR}/")
    print("—" * 50)
```

```
────────────────────────────────────────────────────────────
  ΖΗΤΗΜΑ #4 — Μοντέλο Διαρροής (Hamming Weight)
────────────────────────────────────────────────────────────

  Τιμή   Αναμ.   Υπολ.    OK
0x00         0       0  ✓
0x01         1       1  ✓
0x0F         4       4  ✓
0xFF         8       8  ✓
0x55         4       4  ✓
0xAA         4       4  ✓

Μέσος HW   : 4.000
Τυπ. απόκλ.: 1.414
Αποθηκεύτηκε: figures_project6\fig2_hw_distribution.png

────────────────────────────────────────────────────────────
  Ολοκλήρωση — γραφήματα στον φάκελο: figures_project6/
────────────────────────────────────────────────────────────
```



Συμπεράσματα

Στην παρούσα εργασία μελετήθηκε η διαρροή πληροφορίας μέσω κατανάλωσης ισχύος σε υλοποίηση του αλγορίθμου AES-128. Μέσα από την ανάλυση πραγματικών power traces έγινε εμφανές ότι η κατανάλωση ισχύος εξαρτάται από τις ενδιάμεσες τιμές που επεξεργάζεται η συσκευή. Η οπτικοποίηση των traces έδειξε την παρουσία θορύβου αλλά και κοινών χαρακτηριστικών, γεγονός που καταδεικνύει τη δυνατότητα εφαρμογής στατιστικών επιθέσεων.

Η επιλογή της εξόδου της S-box ως στόχου και η χρήση του μοντέλου Hamming Weight επέτρεψαν τη θεωρητική συσχέτιση μεταξύ δεδομένων και κατανάλωσης ισχύος. Η εργασία αναδεικνύει ότι η ασφάλεια ενός κρυπτογραφικού συστήματος δεν εξαρτάται μόνο από τη μαθηματική ισχύ του αλγορίθμου, αλλά και από τον τρόπο υλοποίησής του σε πραγματικό υλικό. Οι επιθέσεις πλευρικού καναλιού αποτελούν χαρακτηριστικό παράδειγμα του χάσματος μεταξύ θεωρητικής και πρακτικής ασφάλειας.

ΣΤΗΝ ΠΡΑΞΗ:

Ο ΑΕΣ στη θεωρία δεν «σπάει», καθώς μαθηματικά είναι σχεδόν αδύνατο να βρεθεί το κλειδί. Στην πράξη ωστόσο, ο αλγόριθμος αυτός τρέχει πάνω σε μία πραγματική συσκευή, η οποία απαιτεί τροφοδοσία για να λειτουργήσει και επομένως αφήνει ίχνη με βάση το πόση ενέργεια απαιτεί για να υπολογίσει ό,τι της ζητείται. Η CPA εκμεταλλεύεται αυτά τα φυσικά ίχνη και έτσι μπορεί κανείς να πάρει τις πληροφορίες που χρειάζεται για το κλειδί χωρίς να πρέπει να λύσει τον ίδιο τον αλγόριθμο, γι'αυτό και η επίθεση αυτή είναι Side Channel Attack. Επομένως, η θεωρητική ασφάλεια (ο αλγόριθμος ως μαθηματικό μοντέλο) διαφέρει από την πρακτική ασφάλεια (το hardware πάνω στο οποίο τρέχει ο αλγόριθμος).